AWS USER GROUP SANTO DOMINGO

De GitHub a AWS

Descubre cómo construir pipelines de CI/CD modernos integrando GitHub Actions con AWS.

#AWSUGSDQ

¿Quién Soy?



Enmanuel Medina

Cloud Engineer | kubestronaut | 2xAWS | Azure

Enmanuel Medina

Cloud Engineer @ Freelance

- Cloud engineer apasionado por impulsar la **innovación empresarial** a través de soluciones en la nube.
- Co-organizador AWS UG SDQ & CNCF SDQ (K8S)
- @en-medina
- @enmedina3
- @enmedina

Agenda

- Introducción a los pipelines
- GitHub Actions: automatizando nuestro flujo de trabajo
- Autenticación entre GitHub y AWS
- Self-Hosted Runners
- Integraciones con OIDC
- Integraciones con AWS Codebuild
- Demo
- Q&A

Introducción a pipelines: ¿Qué es un pipeline?

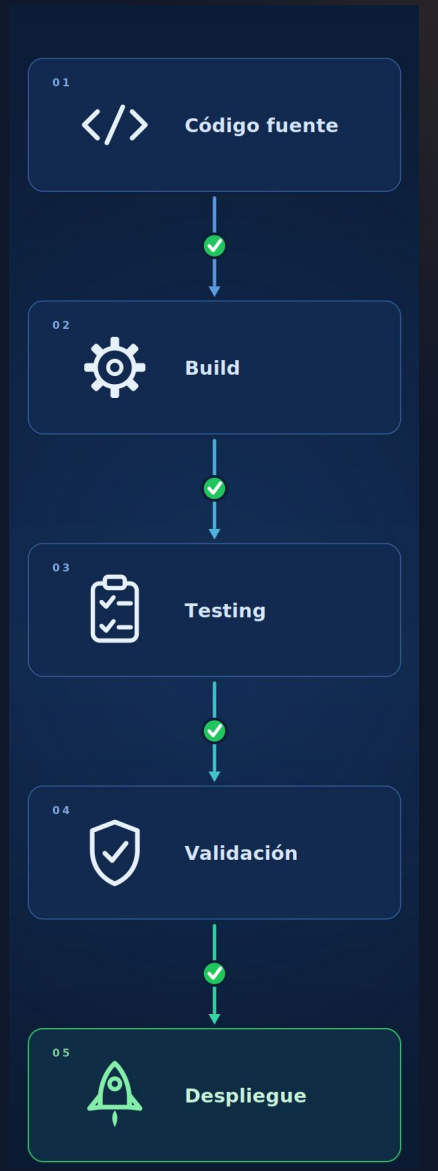
Un pipeline es una secuencia de pasos automatizados que permite construir, validar y entregar software de forma consistente.



Introducción a pipelines: ¿Qué es un pipeline?

Etapas comunes:

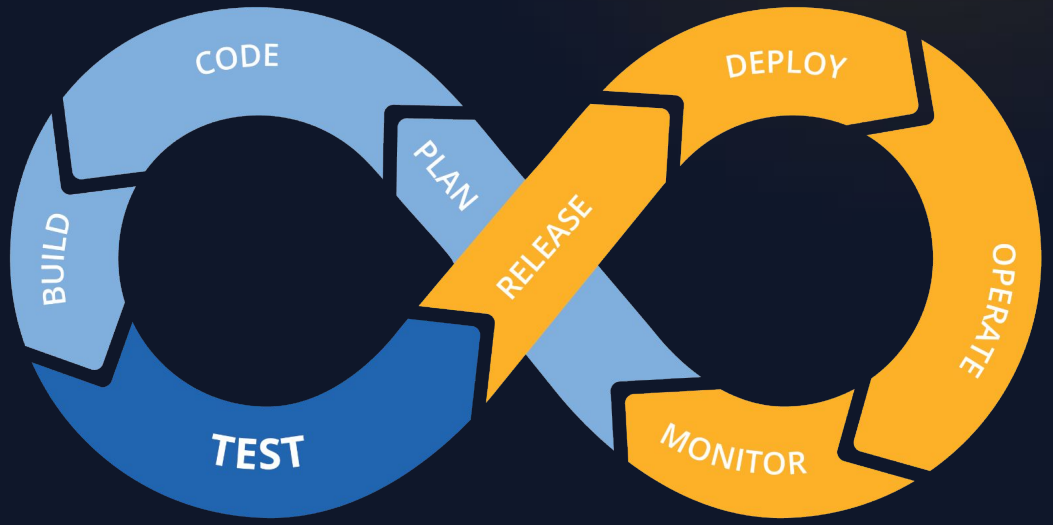
- 📥 Obtener el código fuente.
- 📦 Instalar dependencias.
- 🧪 Ejecutar pruebas.
- 🏗️ Construir y empaquetar la aplicación.
- 🚀 Desplegar en un entorno.



Introducción a pipelines: ¿Qué es CI/CD?

CI/CD reúne prácticas que automatizan la integración y entrega de software.

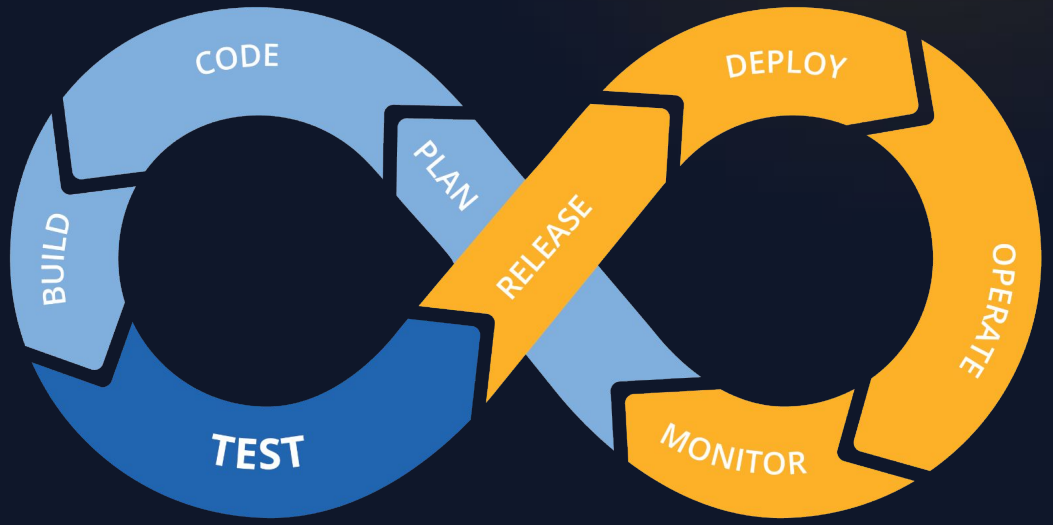
⚙️**Integración Continua (CI)**: integra cambios de código frecuentemente y ejecuta validaciones automáticas.



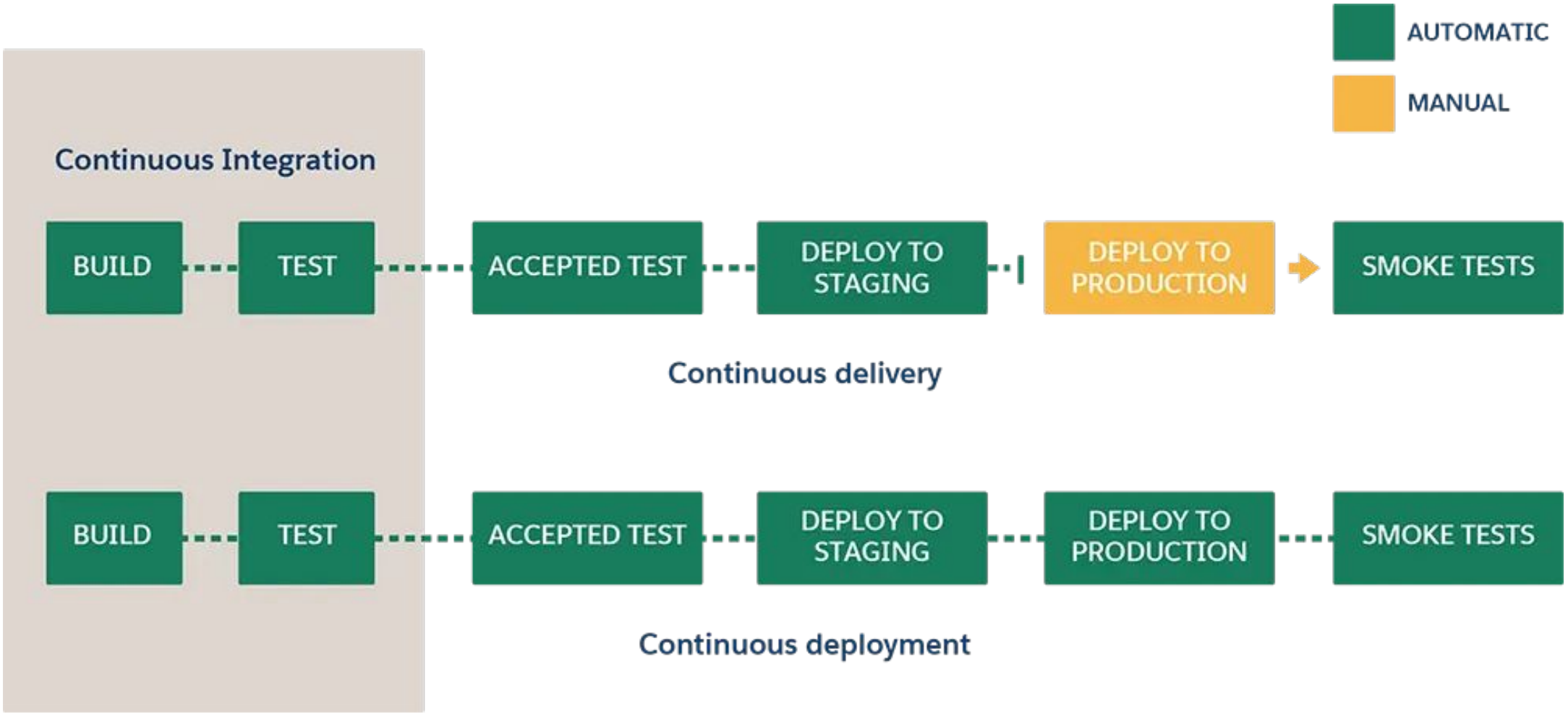
Introducción a pipelines: ¿Qué es CI/CD?

📦 **Entrega Continua (CD):** mantiene el software listo para desplegarse cuando sea necesario.

🚀**Despliegue Continuo (CD):** despliega automáticamente los cambios que superan las validaciones.



Introducción a pipelines: ¿Qué es CI/CD?



GitHub Actions: automatizando nuestro flujo de trabajo

GitHub Actions es una plataforma de automatización integrada con GitHub que permite ejecutar tareas en respuesta a eventos del repositorio.



GitHub Actions: automatizando nuestro flujo de trabajo

Conceptos principales:

📋 **Workflow:** define el flujo automatizado.

⚡ **Event:** determina cuándo se ejecuta.

🛠️ **Job:** agrupa tareas que se ejecutan en un runner.



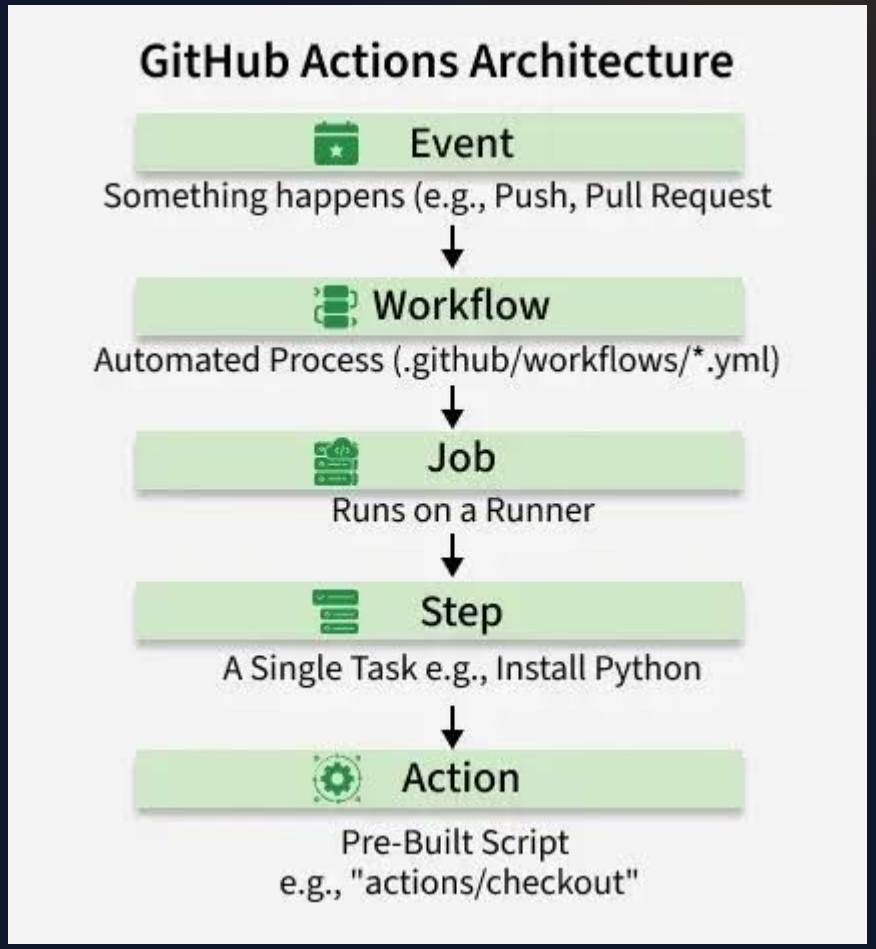
GitHub Actions: automatizando nuestro flujo de trabajo

Conceptos principales:

📌 **Step:** representa una acción o comando.

🖥️ **Runner:** máquina que ejecuta los jobs.

Ejemplos: pruebas al abrir un Pull Request y despliegues al integrar cambios en main.



GitHub Actions: automatizando nuestro flujo de trabajo

sample.yml U WORKFLOW

.github > workflows > sample.yml

```yaml
name: learn-github-actions
run-name: ${{ github.actor }} is learning GitHub Actions
on: [push] EVENT
jobs:
  hello-world: JOB
    runs-on: ubuntu-latest RUNNER
    steps:
      - name: Run a one-line script STEP
        run: echo "Hello, world!"
```

Auth GitHub <> AWS : Por qué es necesario?

Para que un workflow pueda interactuar con AWS, necesita autenticarse y obtener los permisos necesarios.

Esto permite evitar llamadas no autorizadas de terceros a los recursos computacionales que se desean desplegar



Auth GitHub <> AWS : Por qué es necesario?

Hoy se analizará las siguientes opciones:

- Uso de GitHub Secrets para gestionar credenciales.

- Integración con OIDC: autenticación sin credenciales de larga duración.

- Permisos propios de Codebuild (self-hosted)



Auth GitHub <> AWS : Github Secret

GitHub Secrets permite almacenar información sensible para utilizarla dentro de los workflows sin escribirla directamente en el código.



Auth GitHub <> AWS : Github Secret

🔐 Se guardan las credenciales de AWS en los secretos del repositorio o de la organización.

⚙️ El workflow obtiene los valores durante la ejecución.

☁️ Las credenciales se utilizan para configurar el acceso a AWS.



Auth GitHub <> AWS : Github Secret

IMPORTANTE: las credenciales de larga duración requieren rotación, protección y permisos mínimos.



GitHub Secrets protege los valores almacenados, pero **no elimina los riesgos de exposición** durante la ejecución.

Auth GitHub <> AWS : Autenticación mediante OIDC

OIDC (OpenID Connect) permite que GitHub Actions obtenga credenciales temporales de AWS sin almacenar claves de acceso permanentes.



Ventaja principal: reduce la necesidad de almacenar y rotar credenciales permanentes.

Auth GitHub <> AWS : Autenticación mediante OIDC

¿Cómo funciona?

1. GitHub Actions solicita un token de identidad.

2. AWS valida el token mediante un proveedor de identidad OIDC.



Auth GitHub <> AWS : Autenticación mediante OIDC

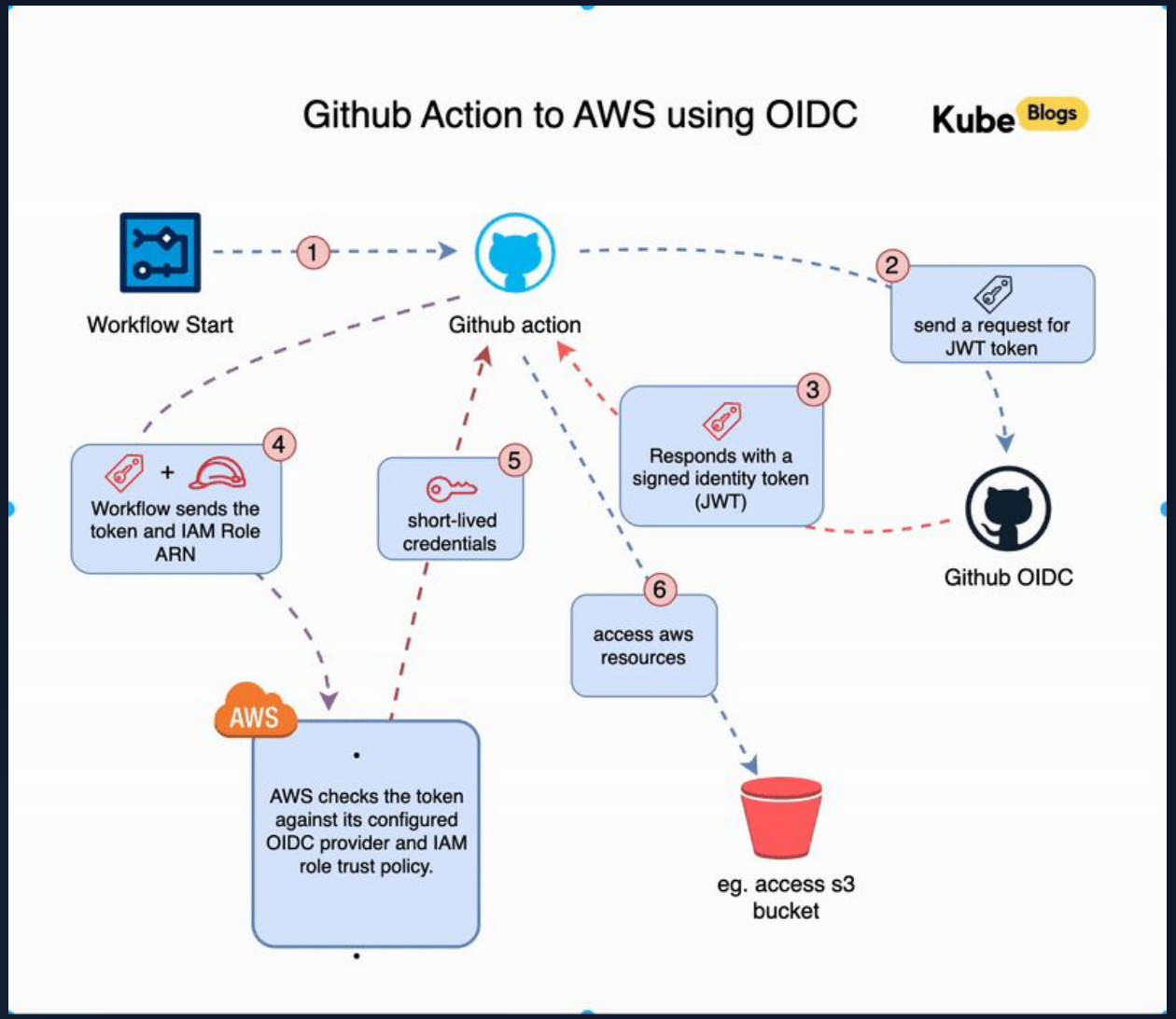
3. Una política de confianza determina si el workflow puede asumir un rol IAM.

4. AWS STS entrega credenciales temporales.

5. El workflow utiliza esas credenciales para acceder a los recursos autorizados.

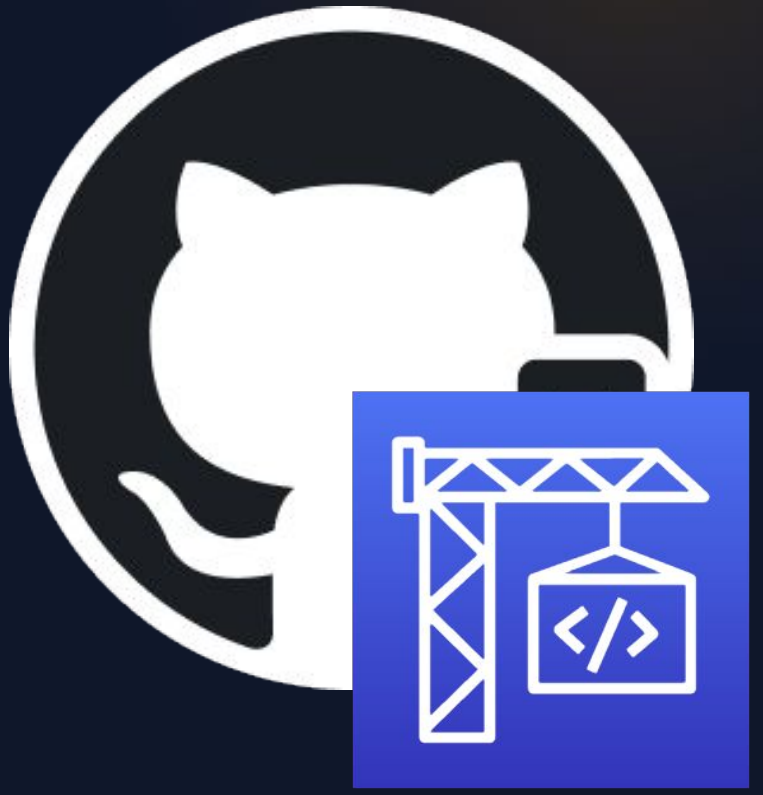


Auth GitHub <> AWS : Autenticación mediante OIDC



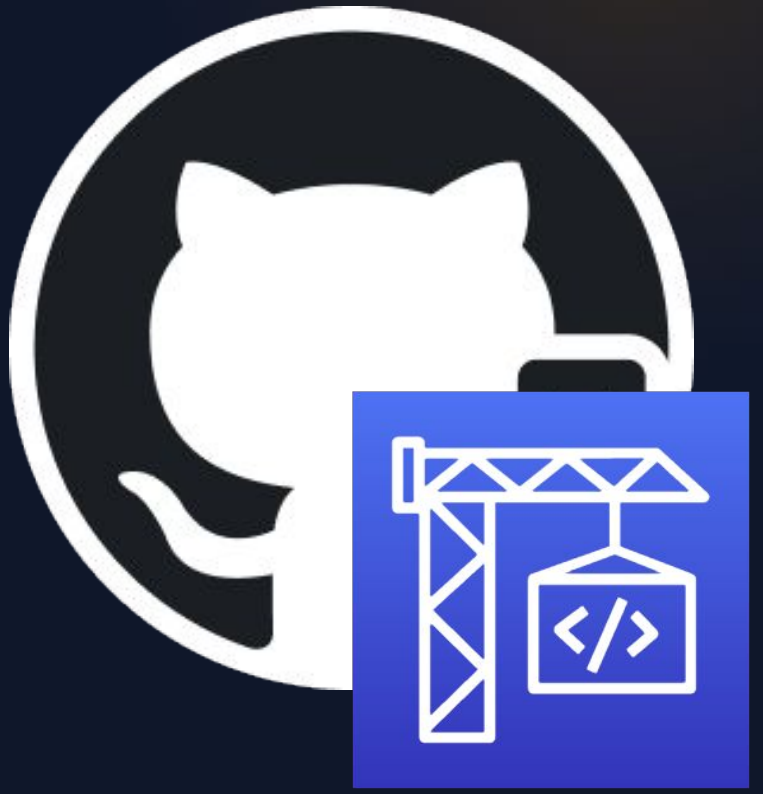
Auth GitHub <> AWS : Permisos propios de Codebuild

AWS CodeBuild puede utilizarse como un Self-Hosted Runner para GitHub Actions, permitiendo ejecutar workflows en infraestructura administrada por AWS sin tener que mantener servidores propios.



Auth GitHub <> AWS : Permisos propios de Codebuild

¿Cómo funciona?



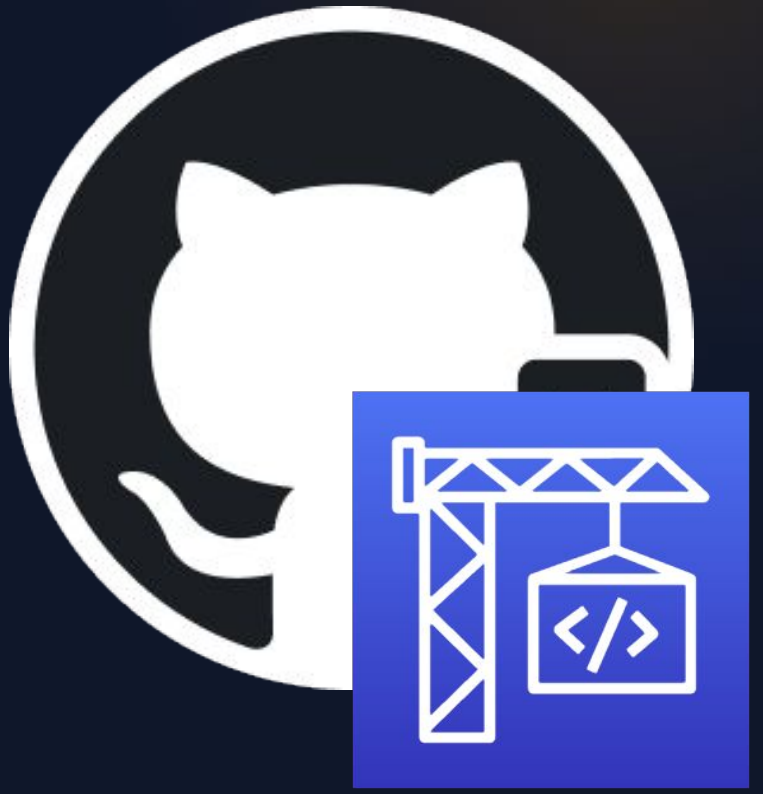
👤 **Service Role:** define los permisos que CodeBuild puede utilizar para interactuar con otros servicios de AWS.

🔐 **Mínimo privilegio:** otorgar únicamente los permisos necesarios para ejecutar el pipeline.

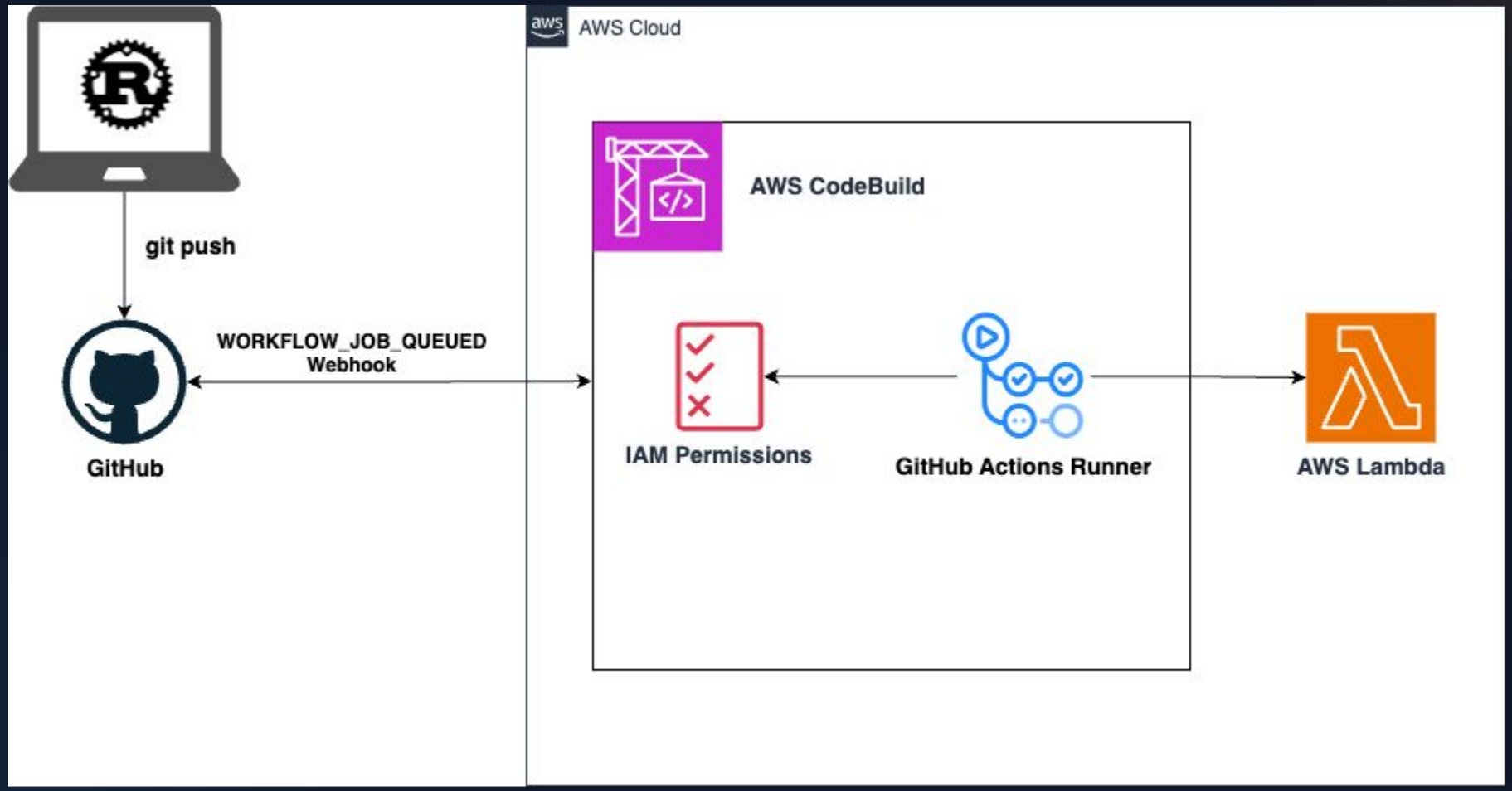
Auth GitHub <> AWS : Permisos propios de Codebuild

🎛️ **Separación de responsabilidades:** distinguir los permisos para ejecutar el build de los permisos para desplegar recursos.

⏱️ Credenciales temporales: CodeBuild obtiene credenciales del rol mediante AWS y las utiliza durante la ejecución.

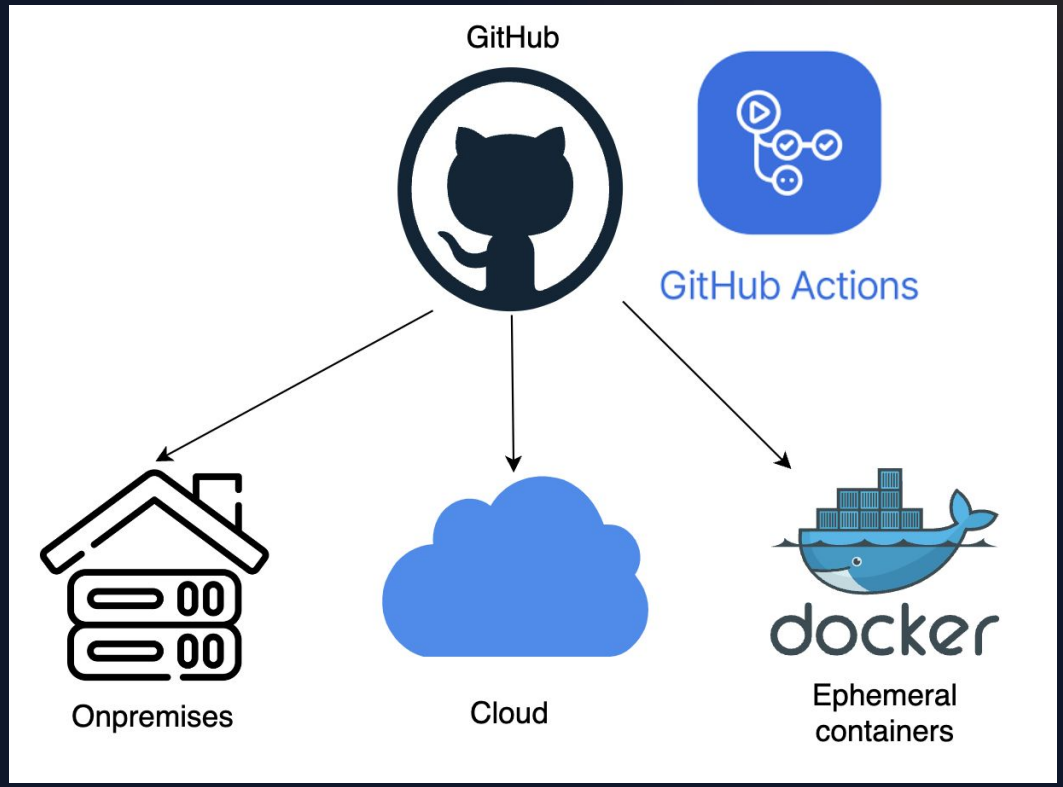


Auth GitHub <> AWS : Permisos propios de Codebuild



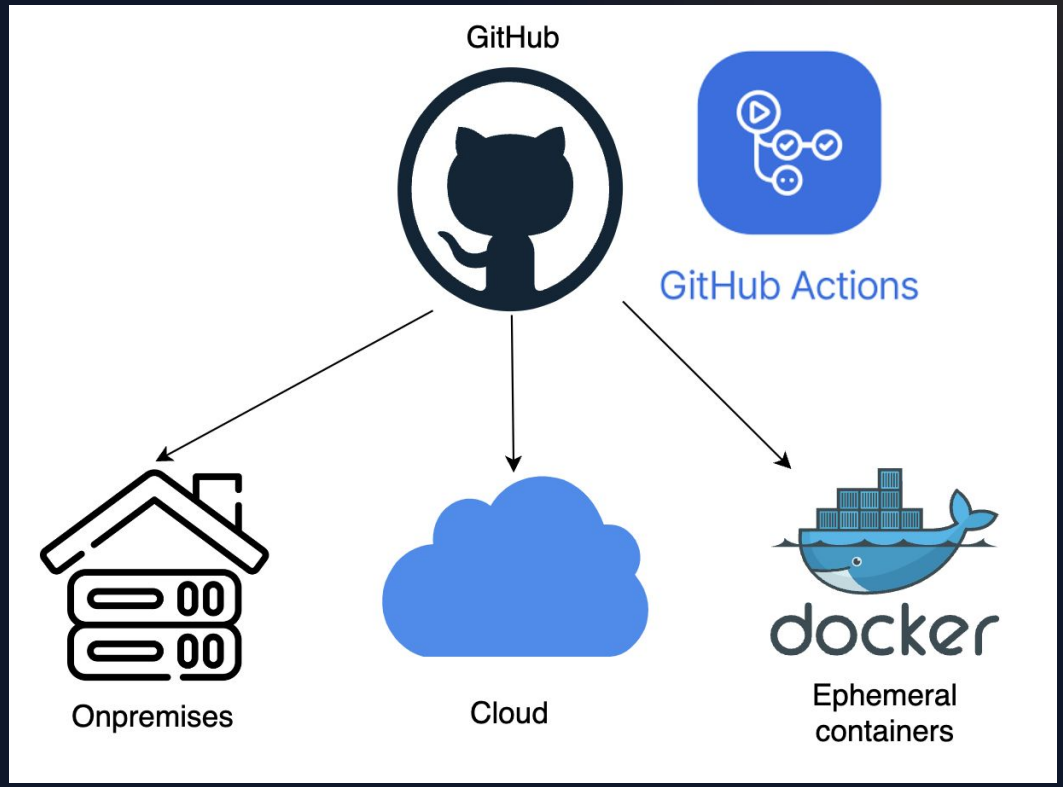
Self-hosted Runner: Definición

Un Self-Hosted Runner es una máquina administrada por nosotros (o un tercero) que ejecuta los jobs de GitHub Actions.



Self-hosted Runner: Definición

Puede ser una máquina virtual, un servidor físico, una instancia EC2 o una infraestructura administrada mediante otros servicios.

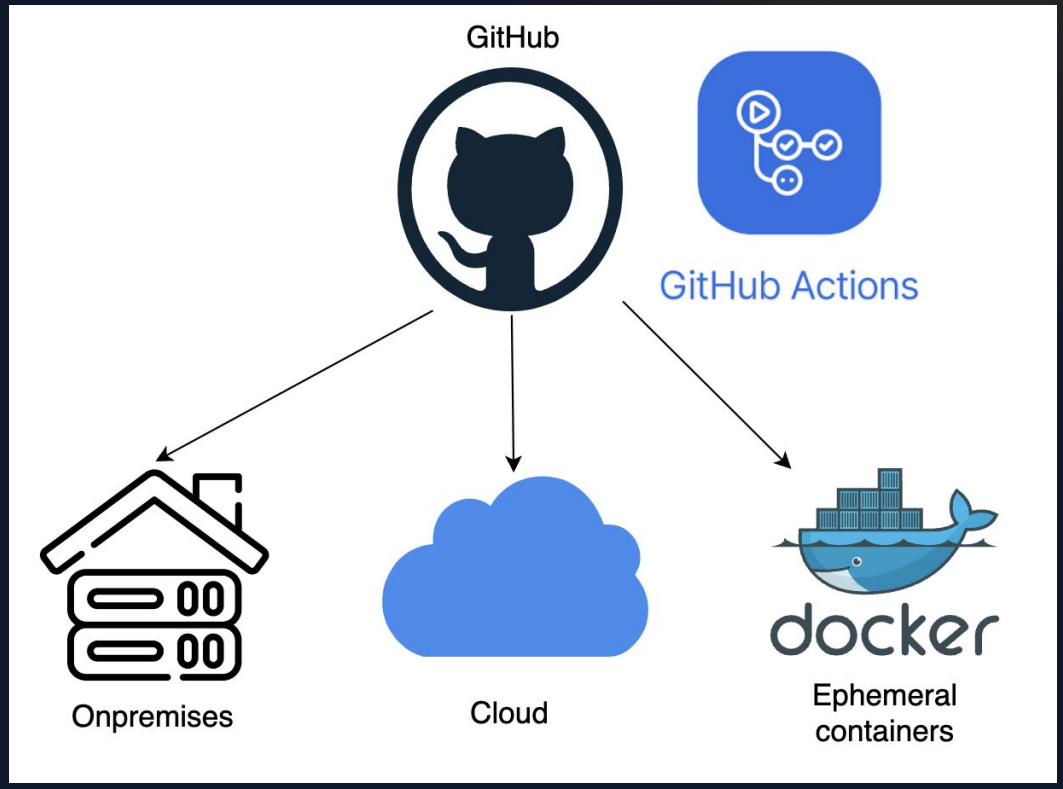


Self-hosted Runner: Definición

Casos de usos:

🧰 Cuando se necesitan herramientas o dependencias específicas.

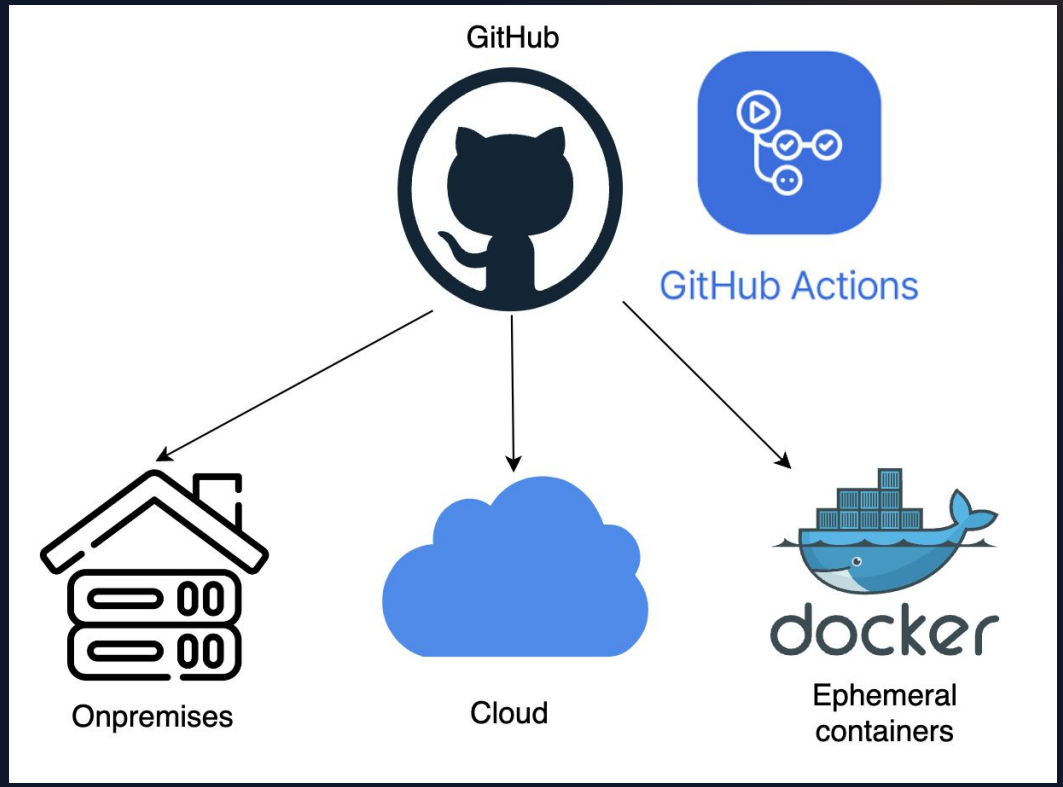
🔒 Para acceder a recursos privados de una red.



Self-hosted Runner: Definición

🎛️ Cuando se requiere mayor control sobre el entorno de ejecución.

⚙️ Para ejecutar procesos que necesitan una configuración particular.



Integración: Github + AWS OpenID Connect

¿Qué necesitamos configurar?

- **GitHub Actions:** permiso id-token: write para solicitar un token OIDC.
- **Proveedor OIDC en AWS IAM:** confiar en el emisor de tokens de GitHub (token.actions.githubusercontent.com).

Integración: Github + AWS OpenID Connect

¿Qué necesitamos configurar?

- **Rol IAM:** definir una política de confianza que determine qué repositorio, rama o entorno puede asumir el rol.

- **Permisos IAM:** especificar qué recursos y acciones de AWS puede utilizar el workflow.

Integración: Github + AWS CodeBuild

¿Qué necesitamos para integrar ambos servicios?

- **Configurar la integración con GitHub Actions:** habilitar el soporte correspondiente en CodeBuild y definir cómo se ejecutarán los workflows.

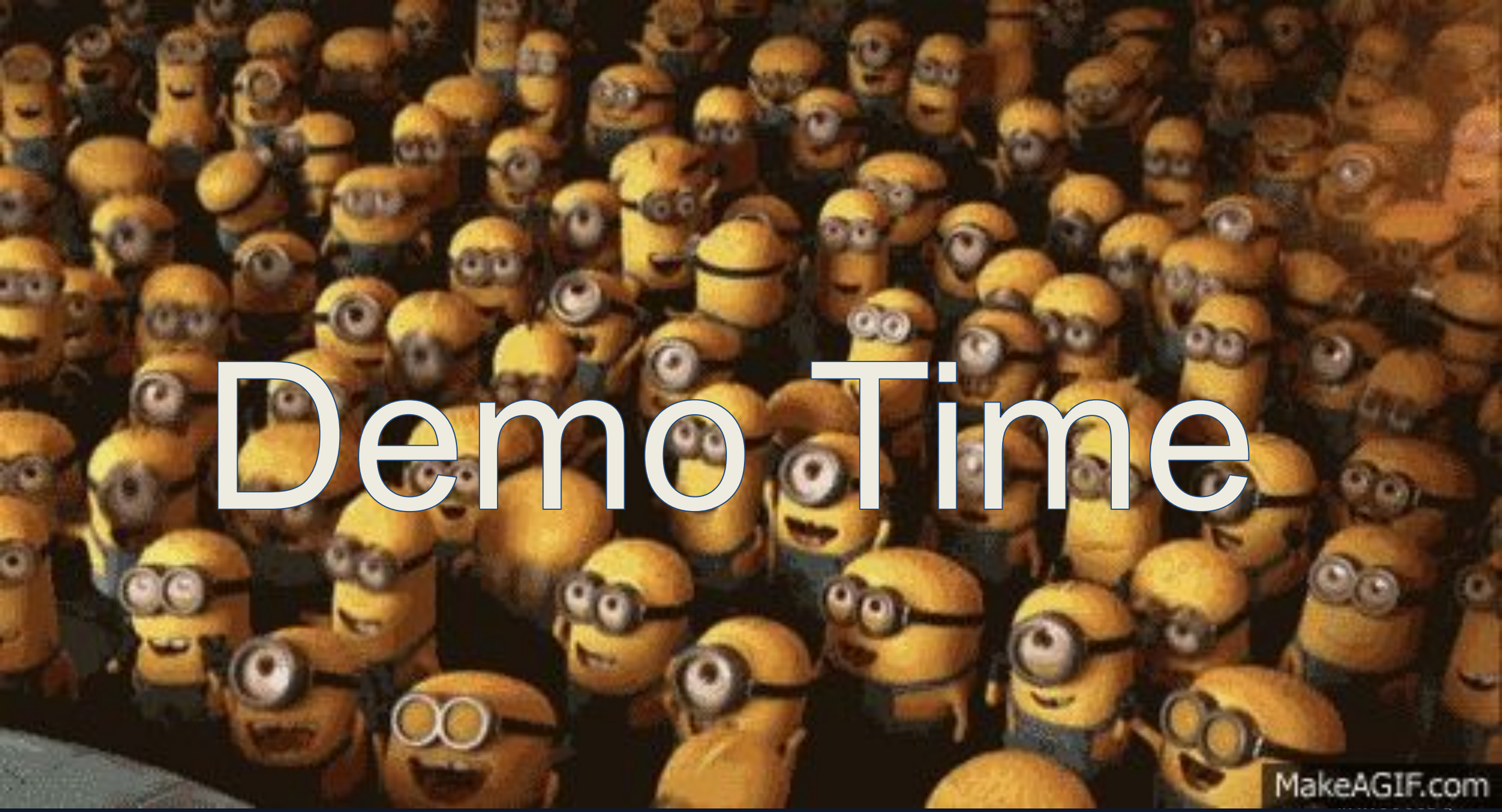
- **Configurar IAM:** asignar a CodeBuild un Service Role con los permisos mínimos necesarios.

Integración: Github + AWS CodeBuild

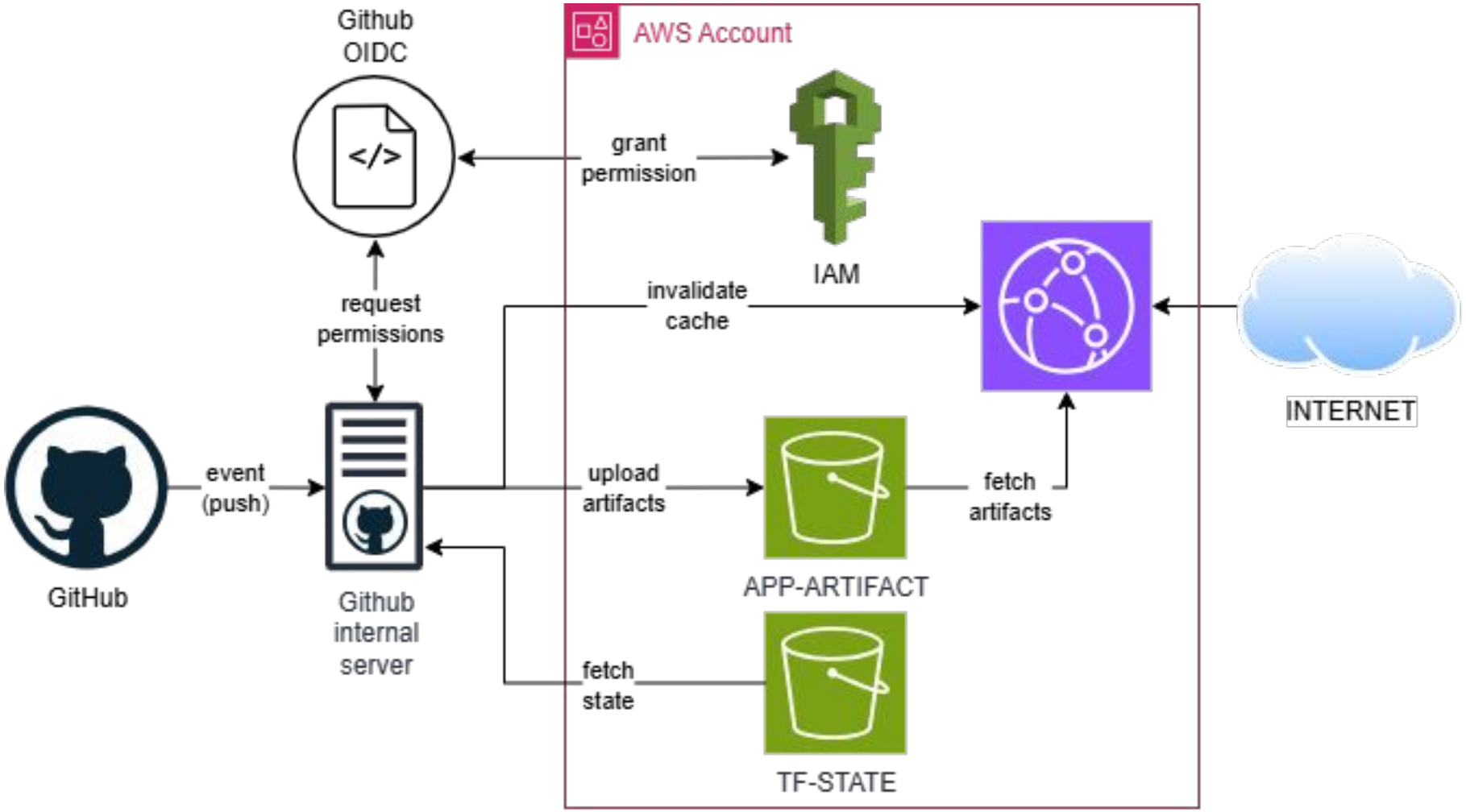
¿Qué necesitamos para integrar ambos servicios?

- **Definir el entorno:** seleccionar imagen de ejecución, recursos de cómputo y configuración del proyecto.

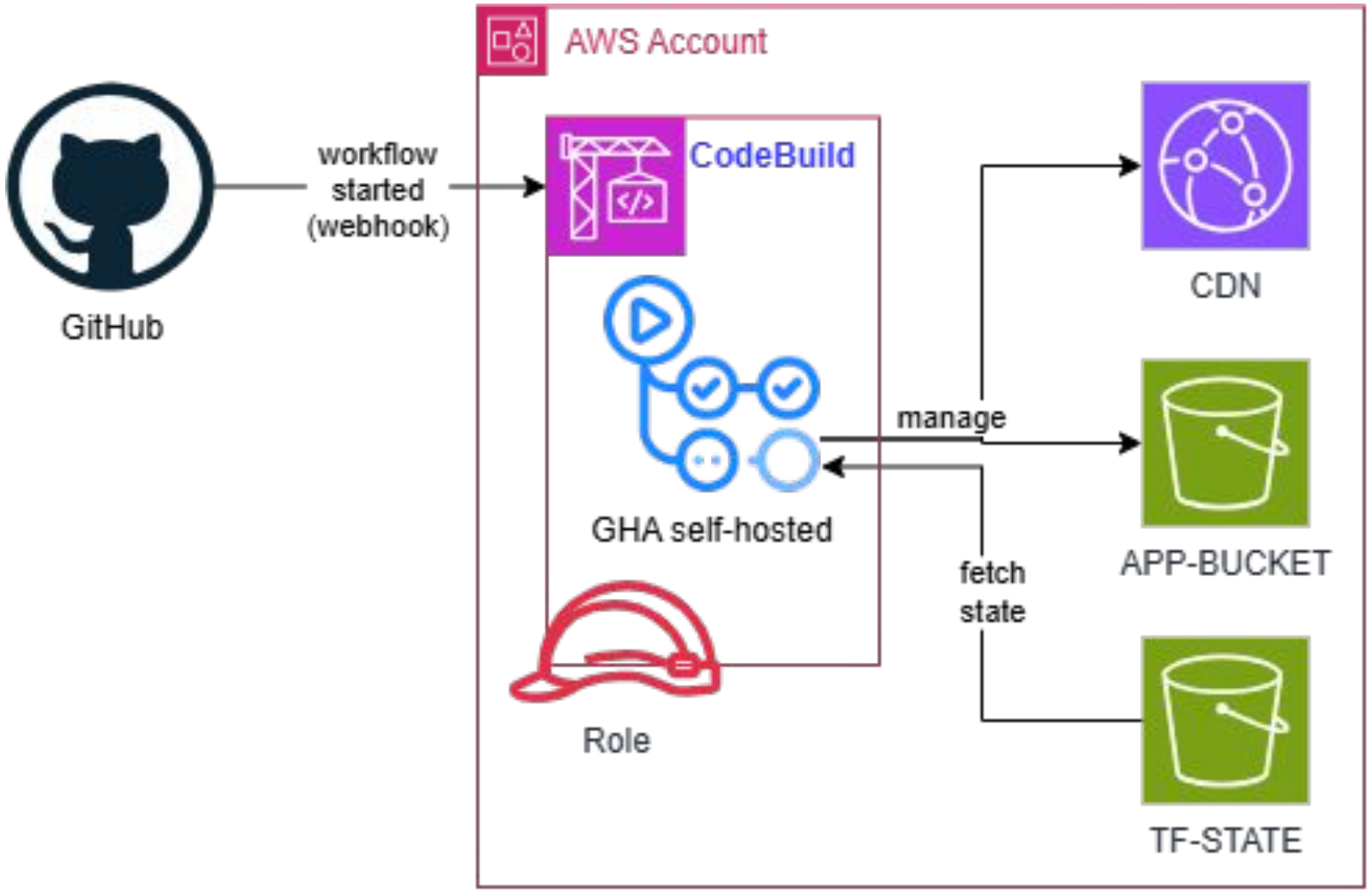
- **Ejecutar y validar:** comprobar que el job se ejecuta correctamente y que solo accede a los recursos autorizados.


Demo Time
MakeAGIF.com

Pero antes de... Diagrama Github <> OIDC <> AWS



Pero antes de... Diagrama Github <> CodeBuild



Demo: Ubicación de código fuente y página web





¿Preguntas?

¡Gracias por acompañarnos y ser parte de la comunidad!

AWS User Group Santo Domingo | @AWSUGSDQ

#AWSUGSDQ